

Chatbot Helpdesk for Centralized Meter Record Application (ACMT) by PLN using Deep Learning

Dari Suparno
Master of Information Technology
Universitas Teknologi Digital
Indonesia
Yogyakarta, Indonesia
student.dari_suparno@mti.utdi.ac.id

Domy Kristomo
Master of Information Technology
Universitas Teknologi Digital
Indonesia
Yogyakarta, Indonesia
domy@utdi.ac.id

Widyastuti Andriyani
Master of Information Technology
Universitas Teknologi Digital
Indonesia
Yogyakarta, Indonesia
widya@utdi.ac.id

Abstract—The Centralized Meter Record Application (ACMT) is a mobile and back-office application used by field meter recording officers. Users often face issues and wait for responses from helpdesk and technical support. This research aims to develop a chatbot model that can assist helpdesk and technical support in answering questions according to standard operating procedures by using deep learning. A chatbot is a computer program designed to interact with humans through text conversations. Deep learning is chosen for chatbot analysis because it excels in understanding and generating human-like language, enabling natural, context-aware, and adaptable interactions. Data is collected from IT Service Management (ITSM) and used for intent creation. TensorFlow is used for machine learning and deep learning, and the model training process uses itertools library to determine the number of neurons in each layer. The model is trained for 90 epochs, achieving an accuracy of 96% with a model size of 209.3 KB. Testing using the Robot Framework showed an accuracy of 80%.

Keywords—centralized meter record application, PLN, deep learning, helpdesk, service desk

I. INTRODUCTION

The increased use of information and communication technology (ICT) has transformed how people interact with various services, including helpdesk or customer service applications. Chatbots are one technology that can be used to provide automated services to application users. Chatbots can be programmed to answer questions and resolve user issues independently. The Centralized Meter Record Application (ACMT) by the National Electricity Company (PLN) is one such application that can facilitate users in recording meter readings for customers. This application is used by officers to record customers' meter usage and conditions. Meter reading officers typically report complaints through a service desk, which are then addressed by the helpdesk. With the presence of a chatbot, it is expected that officers encountering issues will receive immediate responses from this AI chatbot, leading to quicker resolutions. Developing an effective and responsive chatbot for the ACMT requires a careful approach and the right technology. One promising approach is the use of Deep Learning, specifically with frameworks such as TensorFlow and Keras. Deep Learning enables the chatbot to learn from existing data and improve its ability to understand natural language and respond to user queries or requests more accurately. Developing a helpdesk chatbot for the ACMT using Deep Learning faces several challenges that need to be addressed. These include a deeper understanding of user needs, proper integration with the Centralized Meter Record

system, and ensuring user data security and privacy in interactions with the chatbot.

This research aims to investigate and develop an effective and responsive helpdesk chatbot for the Centralized Meter Record Application (ACMT) using a Deep Learning approach. The expected outcome of this research is to enhance user experience and service efficiency in the ACMT by PLN, and to contribute to the broader development of chatbot technology.

II. METHODOLOGY

A. Chatbot and Deep Learning

A Chatbot is a software application that helps in conducting a conversation by using auditory or text-based methodologies. The programs in a Chatbot are developed to imitate human conversations. The use of Chatbot's for various purposes include customer services, request routing, or for information gathering. Some chatbots are extensively used for word-classification processes by using Natural language processing [1]. Deep learning is a subfield of machine learning whose algorithms are inspired by the structure of the human brain. This structure is called Artificial Neural Networks (ANN). Essentially, it is a neural network with three or more ANN layers. It is capable of learning and adapting to large amounts of data and solving various problems that are difficult to address with other machine learning algorithms [2].

B. Natural Language Processing (NLP) and Natural Language Toolkit (NLTK)

Natural Language Processing is the program interface of how computers and humans interact with each other. This technique allows computer programs to analyze huge amounts of language data from various resources. The NLP developers can organize the knowledge to perform various tasks such as analyze text, text summarizing, topic extraction, stemming, text mining, automatic summarization, translation, speech recognizing, segmentation, and automated question answering [3]. Natural Language Toolkit is mainly programmed for building and applying symbolic and statistical NLP in python. It is a suite of libraries including text processing for tokenization, parsing the text, classification of text, stemming of words, and reasoning of semantics [4].

C. Keras and Tensorflow

Keras is a neural network API library, in python. It runs on top of It is effective in running on top of R, PlaidML, TensorFlow, Microsoft Cognitive Toolkit [4]. Tensorflow is a software library framework that mainly focuses on machine learning that uses dataflow graphs and differentiates programming over various number tasks to build models. It is used by the developers to create large-scale development applications including the neural networks. It is mainly applied for Classification, Understanding, Prediction, and Creation [5].

D. Itertools and FastAPI

Using itertools in Python can be very helpful in finding the optimal number of neurons for a neural network model, especially when trying different parameter combinations (hyperparameter tuning). itertools is a Python module that provides various functions to create efficient iterators, which are useful in parameter exploration [6]. This framework is designed to optimize the development experience so that simple code can then be written to build production-ready APIs with best practices by default [7].

E. Confusion Matrix

A confusion matrix is a tool used to test a model's ability to detect or correct errors it makes. It measures the True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN) of a model's predictions. Additionally, a confusion matrix can provide evaluations of performance metrics such as accuracy, precision, recall, and F1-score [8].

III. LITERATURE REVIEW

Literature studies are carried out by studying related previous research obtained from various sources such as books, journal publications, papers, theses, dissertations and others. For the literature study, we took a journal from [9] entitled "Implementation of Artificial Neural Network Algorithm in Chatbot Development Using Natural Language Processing Approach".

In this paper, the researchers implemented an Artificial Neural Network algorithm to create a chatbot using a Natural Language Processing approach. The data structure used in this research is a set of intents (dataset) manually entered and stored in a JSON (JavaScript Object Notation) file. The dataset structure includes:

1. Intents: All the input and output data used to train the chatbot
2. Tags: Group similar text data and use the same as the target output for training the neural network.
3. Patterns: The section containing the input pattern data desired by the user.
4. Responses: The section containing the output pattern data that the chatbot will send to the user.

From this data, the processing will involve the following five stages:

1. Case Folding
2. Tokenizing
3. Filtering
4. Stemming
5. Bag Of Word

An Artificial Neural Network (ANN) is a computational algorithm that can solve complex problems by imitating the brain's processes in a simplified manner. It consists of artificial neurons or nodes, which are information processing units arranged in layers and interconnected with synaptic weights/connections. The ANN architecture model is a three-layer network of interconnected nodes, namely the input layer, hidden layer, and output layer. Each layer in an ANN contains an activation function. The activation function determines the output of a neuron, which can be linear or nonlinear. There are several activation functions such as ReLU (Rectified Linear Unit), Softmax, Sigmoid, Linear, and others. The activation functions used in this research include:

1. ReLU Activation Function

The ReLU (Rectified Linear Unit) function is an activation function with a simple computation. If an element has a negative value, it is changed to 0, and there are no exponential, multiplication, or division operations involved. The ReLU activation function is formulated in equation (1) and visualized in the Figure 1 below.

$$f(x) = \max(0, x) \text{ atau } f(x) = \begin{cases} 0 & \text{untuk } x \leq 0 \\ x & \text{untuk } x > 0 \end{cases} \quad (1)$$

This equation and visualization illustrate how ReLU activation works by setting all negative values to zero while leaving positive values unchanged.

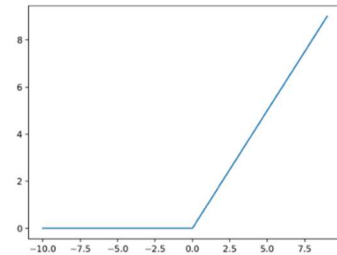


Fig. 1. Chart ReLU Activation Function ANN Architecture with 4 layer.

2. Softmax Activation Function

The softmax activation function is used in probability calculations to determine multi-class classification by selecting the class with the highest probability value. The output produced by the softmax activation function has probability values between 0 and 1. The softmax activation function is formulated in equation (2) and visualized in the Figure 2 below.

$$f(X_i) = \frac{\exp(X_i)}{\sum_{j=0}^k \exp(X_j)}, \text{ nilai } i = 0, 1, 2, \dots, k \quad (2)$$

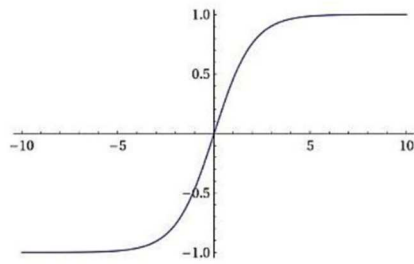


Fig. 2. Chart Softmax Activation.

IV. PROPOSED WORK

A. Design

At this stage, the system design process is carried out. Figure 3 illustrates the training process, starting with data retrieval from the ITSM application and then exporting the data to produce an intents file. After that, it goes through the NLP process, which consists of Tokenizing, Lemmatization, Case Folding, and Deduplication, resulting in a Bag of Words. Finally, the data created is used in the training process, producing three files: class file, word file, and model.

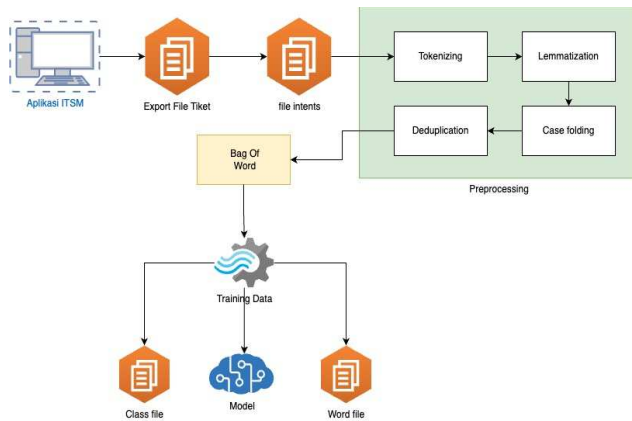


Fig. 3. Training process.

The ITSM (IT Service Management) application is an application that accommodates all complaints related to applications, which will be followed up by Helpdesk or IT Support.

The Export File Ticket feature in ITSM allows the selection of Service Families to be exported in Excel format. This feature also categorizes questions and provides appropriate answers.

Bag of Words is an approach in Natural Language Processing (NLP) used to represent text numerically. This approach disregards the structure and order of words in the text and focuses only on the presence of words within a document. The Bag of Words process allows us to transform text into a numerical representation that can be used as input for machine learning algorithms. However, the main drawback of Bag of Words is the loss of word order information and the lack of consideration for the contextual meaning between words. Some enhancements, such as TF-IDF (Term Frequency-Inverse Document Frequency), can be used to address these shortcomings.

- Training is the process of training a model using data that has been transformed into a Bag of Words representation.
- A model is the result of training a dataset and produces a model that is ready to be used for chatbot applications.
- A class file is a collection of labels or data categories created during training.
- A word file is a collection of words obtained from the dataset used during training.

In Figure 4, the process of a chatbot is illustrated, consisting of Natural Language Understanding (NLU) and Natural Language Generation (NLG) stages. When the chatbot receives input and produces output, the process is as follows:

- Frontend Chatbot Application: The input is first received by the chatbot's frontend application.
- REST API: The input is then sent to a REST API.
- Natural Language Understanding (NLU): The NLU processes the input to understand its meaning.
- Model: The processed input is fed into a model, which outputs an index or category.
- Natural Language Generation (NLG): The index or category is then used in the NLG process to generate a response that aligns with the category provided by the model.

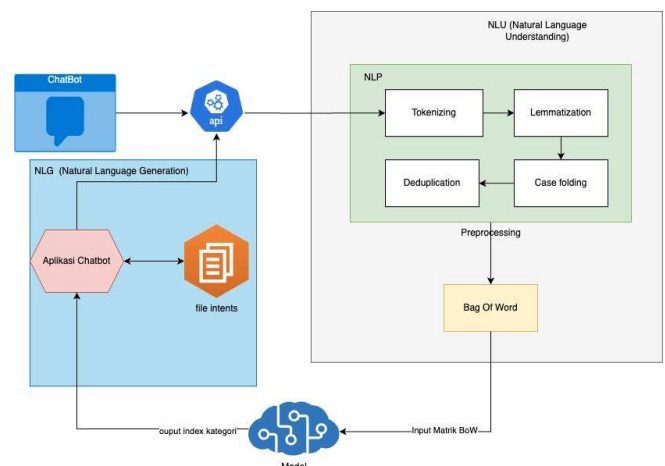


Fig. 4. Chatbot Process

V. RESULTS AND DISCUSSION

1. File Excel form ITSM

- Query data Form ITSM

Perform a query on the ITSM application with the following filters: service group ACMT, ticket status closed, and date range from January 1, 2023, to March 30, 2023

- Export to Excel

Perform a query on the ITSM application with filters for service group ACMT, ticket status closed, and the date range from January 1, 2023, to March 30, 2023. After obtaining the query results, export the data into Excel format.

2. Intens File

The next step is to process the data by assigning categories or category names to similar questions. Move the data from Excel into the Intents.json file.

3. Training Model

To achieve an optimal model in machine learning, especially in terms of selecting the number of layers and neurons, as well as the appropriate hyperparameters, the following steps are taken:

- Finding the Number of Layers and Neurons

Use a total of 4 layers: 1 input layer, 2 hidden layers, and 1 output layer with softmax activation as shown in Figure 5.

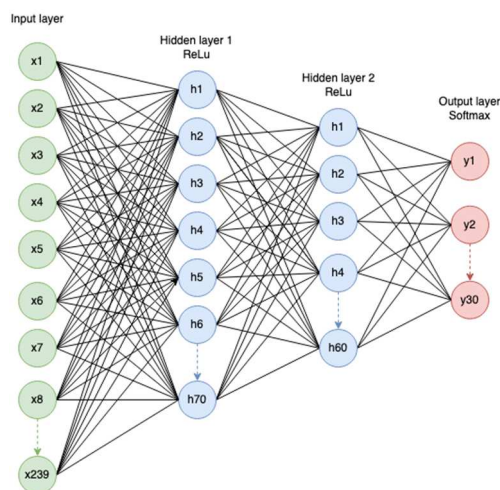


Fig. 5. ANN Architecture

- Get hyperparameter values

Determine the accuracy criteria, with a minimum accuracy of 80 to 90%. Optimize the model size by selecting the combination of hyperparameters that provides an accuracy above 0.9 with the smallest model size. Collect all training results that meet the accuracy criteria above 0.8 or 80% as shown in Table 1.

TABLE I. RESULTS WITH HYPERPARAMETERS

Layer 1	Layer 2	Layer 3	Epoch	Accuracy	Size File (KB)
100	60	30	100	0.9797979593	279.453125
120	60	30	95	0.9797979593	326.171875
70	60	30	90	0.9696969986	209.375
100	60	30	85	0.9696969986	279.453125
110	60	30	100	0.9696969986	302.8125
120	50	30	95	0.9696969986	314.453125
120	60	30	80	0.9696969986	326.171875
110	60	30	70	0.9595959783	302.8125
110	60	30	75	0.9595959783	302.8125
120	60	30	100	0.9595959783	326.171875
90	60	30	95	0.9494949579	256.09375

1. Class Distribution in Training Data

The graph in Figure 6 shows that there are 2 classes with more than 10 samples. This data appears to be imbalanced because some classes have a significantly higher number of samples compared to others. Data balance is an important factor in machine learning models, as imbalance can lead to bias in the model.

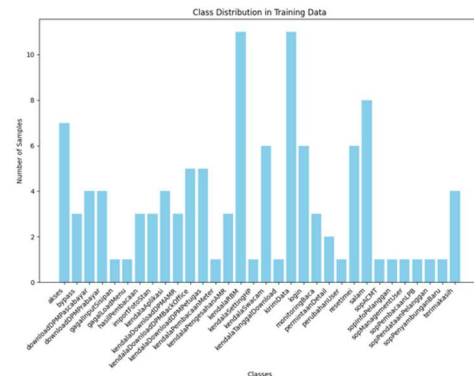


Fig. 6. Chart Class Distribution Training

2. Model Performance During Training

Figure 7 displays a comparison graph of accuracy and loss during training. It shows that as the number of epochs increases, the loss also increases but does not improve the accuracy.

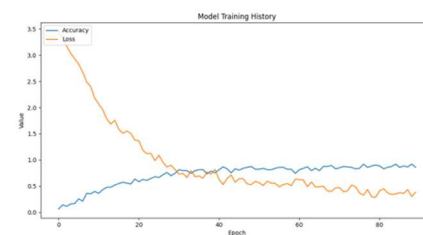


Fig. 7. Performance Model

3. Evaluation Model

To evaluate the model, the researcher visualizes the Confusion Matrix using the following steps: creating a prediction function for the chatbot, collecting prediction data and actual labels, and then creating and displaying the Confusion Matrix as shown in Figure 8.

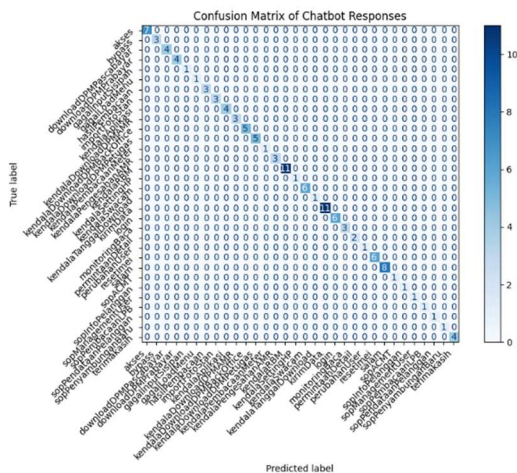


Fig. 8. Confusion Matrix.

4. Create chatbot application

From the training results, a model ready for use in this research has been obtained. This model is implemented using a REST API with the FastAPI framework version 0.110. It is expected that this REST API will facilitate the use of the model across various applications. Figure 9 illustrates the block diagram of the chatbot API application that has been created.

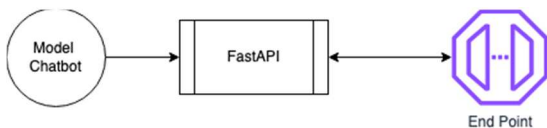


Fig. 9. Diagram Chatbot API.

The result of creating this chat API is a service that can be used by any application endpoint with API-KEY security. Figure 10 shows the testing of the Chatbot API using the POSTMAN application, displaying the results of data submission, the response received, and the response time of 134 milliseconds (ms). This result indicates that the Chatbot API has been successfully created and is ready to be used by other applications. FastAPI also provides automatic documentation accessible to users, facilitating integration. This documentation can be accessed directly via Swagger at the path /docs, as shown in Figure 11.

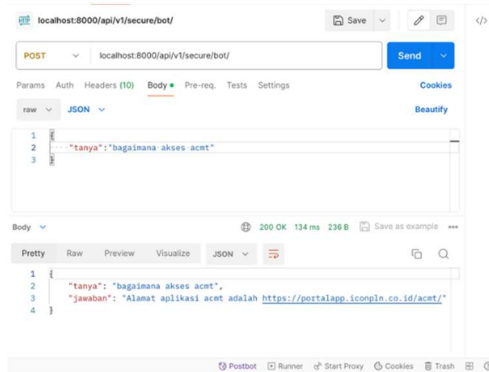


Fig. 10. Postman testing.

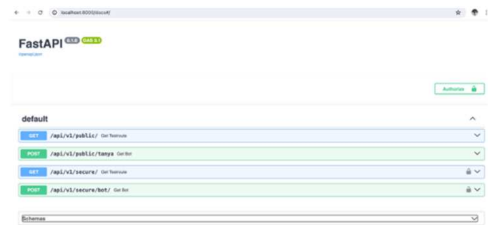


Fig. 11. Swagger UI.

A user interface for the chatbot has also been created using Python with the Tkinter library, as shown in Figure 12. This interface has been tested for submitting questions.

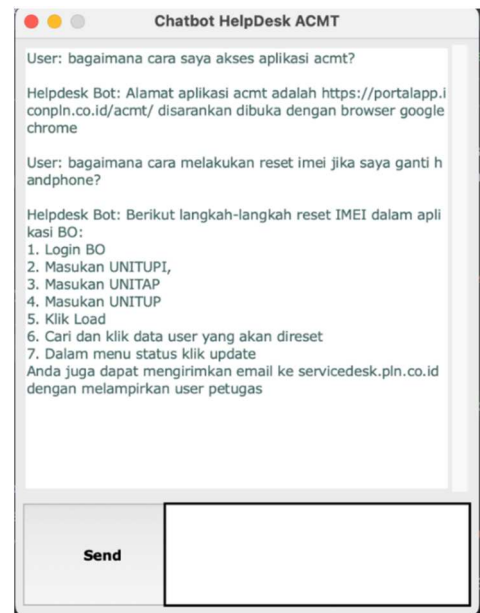


Fig. 12. User Interface Chatbot.

5. Application Testing

For application testing, the Robot Framework is used to perform a loop of questions by creating a list of questions in JSON format, as shown in Figure 13.



Fig. 13. Json file Testing.

Chatbot Testing Log

Generated 20240329 21:30:08 UTC+07:00
117 days 13 hours ago

Test Statistics

Total Statistics	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
All Tests	1	1	0	0	00:00:03	

No Tags

Statistics by Tag	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
Chatbot Testing	1	1	0	0	00:00:03	
Chatbot Testing - Testing	1	1	0	0	00:00:03	

Test Execution Log

Fig. 14. Recap result testing.

Figure 14 summarizes the testing results, showing that all requests were successful with no failures and that it took 3 seconds to request the data. Testing with sentences different from the training data yielded the following results: out of 30 questions, 24 were answered correctly and 6 were answered incorrectly. The success rate is summarized in Table II as follows:

$$\text{Accuracy} = \frac{\text{Number of Correct Tests}}{\text{Total Tests}} \times 100\%$$

$$\text{Accuracy} = \frac{24}{30} \times 100\%$$

$$\text{Accuracy} = 80\%$$

$$\text{Error Rate} = \frac{\text{Number of Incorrect Tests}}{\text{Total Tests}} \times 100\%$$

$$\text{Error Rate} = \frac{6}{30} \times 100\%$$

$$\text{Error Rate} = 20\%$$

TABLE II. TESTING RESULT

No	Class/ tags	Question	Answer	Result
1	salam	halo helpdesk mohon bantuan	Selamat datang di Chatbot Helpdesk ACMT, Terima kasih telah menghubungi kami, silahkan berikan pertanyaan tentang kendala bapak ibu kekami	True
2	terimakasih	terima kasih atas bantuannya	Terima kasih telah menggunakan fasilitas chatbot kami semoga membantu	True
3	akses	alamat URL aplikasi acmt ?	Alamat aplikasi acmt adalah https://portalapp.iconpln.co.id/acmt/	True
4	login	Bagaimana saya agar bisa login di aplikasi acmt	silahkan masukan Username dan password dengan benar	True

VI. CONCLUSION AND RECOMMENDATIONS

This study aims to create an efficient and responsive helpdesk chatbots for the ACMT using a Deep Learning approach. From the experiment, it can be concluded that:

1. Complaint data through ITSM can be used as instances to create a chatbot model.
2. Deep Learning and Natural Language Toolkit can be utilized to build a chatbot model using classification methods.

3. The number of neurons in each layer and the epoch value affect the model size and accuracy.
4. To optimize the model size, the Itertools library can be used.
5. The more detailed the problem categories, the better the classification accuracy.
6. The testing results showed an accuracy of 80% with 30 questions.
7. The REST API application facilitates other applications in accessing chatbot data.

Suggestions for further research that can be done include:

1. Additional problem categories should be included to address a wider range of issues.
2. The chatbot should ideally be able to access a database to handle data requests.
3. With sample conversational data, a Generative AI model could be developed for improved responses.
4. The chatbot model could be designed for offline use on mobile applications, so users do not need an internet connection.

ACKNOWLEDGMENT

This work is partially funded by the research contract number: L.05.1/230/UTDI/PS/VI/2024.

REFERENCES

- [1] S. H. Bariyah and K. A. N. Imania, "Pengembangan Virtual Assistant Chatbot Berbasis Whatsapp Pada Pusat Layanan Informasi Mahasiswa Institut Pendidikan Indonesia - Garut," JURNAL PETIK, vol. 8, no. 1, pp. 66–79, Mar. 2022, doi: 10.31980/jpetik.v8i1.1575.
- [2] R. Setiawan, "Mengenal Deep Learning Lebih Jelas," 2021. Accessed: Jul. 22, 2024. [Online]. Available: <https://www.dicoding.com/blog/mengenal-deep-learning/#:~:text=Apa%20itu%20deep%20learning?,dengan%20algoritma%20machine%20learning%20lainnya>.
- [3] F. Rumaisa, Y. Puspitarani, A. Rosita, A. Zakiah, and S. Violina, "Penerapan Natural Language Processing (NLP) di bidang pendidikan," Jurnal Inovasi Masyarakat, vol. 1, no. 3, pp. 232–235, Dec. 2021, doi: 10.33197/jim.vol1.iss3.2021.799.
- [4] F. Bagwan, R. Phalnikar, and S. Desai, "Artificially intelligent health chatbot using deep learning," in 2021 2nd International Conference for Emerging Technology, INCET 2021, Institute of Electrical and Electronics Engineers Inc., May 2021. doi: 10.1109/INCET51464.2021.9456195.
- [5] D. Dahman, "TensorFlow.Keras," <https://medium.com/sysinfo/tensorflow-keras-66dd489ae52f>.
- [6] R. Shah, "Tune Hyperparameters with GridSearchCV," <https://www.analyticsvidhya.com/blog/2021/06/tune-hyperparameters-with-gridsearchcv/>.
- [7] S. Ramirez, "Menggunakan FastAPI untuk Membangun API Web Python," <https://realpython.com/fastapi-python-web-apis/#reader-comments>.
- [8] Giga Razki Ardiansyah, M. Musayyanah, Wega Aqvirandy, M. Difa Farady, Mia Nilam Cahya, and Tri Hadiono, "Deteksi kanker payudara menggunakan artificial neural network pada deep learning," INFOTECH : Jurnal Informatika & Teknologi, vol. 4, no. 2, 2023, doi: 10.37373/infotech.v4i2.902.
- [9] N. Agustina Purwitasari and M. Soleh, "Implementasi Algoritma Artificial Neural Network Dalam Pembuatan Chatbot Menggunakan Pendekatan Natural Language Processing (Implementation Of Artificial Neural Network Algorithm In Chatbot Development Using Natural Language Processing Approach)," 2022.